DSA5208 Project 3

Hybrid MPI + OpenMP for a 2D Poisson Solver

Jacobi iteration, domain decomposition, and measured CPU scaling on NUS Vanda

Team member	Student ID
Yikun Zhao	A0353894M
Jiahao Zhou	A0354090J
Zheke Chen	A0356140M

NUS DSA5208 | Final project report | Experiments: 3 November 2026

BEST STRONG SPEEDUP	MEDIAN SOLVE TIME	16-CORE EFFICIENCY
13.31x	1.073 s	83.2%

Abstract

We implement a serial C++17 Jacobi solver and parallel versions using MPI row slabs, OpenMP row loops, and two halo exchange methods. The complete mixed validation matrix passes 294 cases and five negative cases on Vanda. At N=2048 and 2600 fixed iterations, the fastest tested strong-scaling configuration is r2t8, with median solve time 1.073 s and 13.31x speedup over the serial executable. Every configuration has one warmup and five measured repetitions. The experiments compare core scaling, weak scaling, rank/thread balance, and blocking versus nonblocking communication on one CPU node.

These are measured results from a completed PBS allocation. Fixed-iteration workloads test execution efficiency; a separate small-grid convergence experiment tests time to solution. No real dataset is used.

Keywords: Jacobi iteration; domain decomposition; halo exchange; hybrid parallelism; strong and weak scaling.

Provenance and code availability

Performance PBS job: 1514382.stdct-mgmt-02. Validation PBS job: 1514325.stdct-mgmt-02. Fixed source revision: af45f5bd84103838cef7e9e5ecdf221585b9fad3. All per-run commands, standard output, standard error, verdicts, source hashes, and executable hashes are archived with the report source.

Planned submission repository: github.com/ZYKuni/DSA5208_project3_submission. This report is accompanied by a self-contained source package; repository migration is pending.

1. Algorithm and numerical contract

We solve $-\Delta u = f$ on $\Omega = (0,1)^2$ with $u = 0$ on the boundary. The grid has N x N interior points and spacing $h=1/(N+1)$. A five-point finite difference stencil gives a sparse linear system. Jacobi updates each interior value using only the previous iterate, so the update is easy to parallelize locally but requires consistent neighbor values across MPI subdomains.

$$u^{(k+1)}_{i,j} = ¼ [u^{(k)}_{i-1,j} + u^{(k)}_{i+1,j} + u^{(k)}_{i,j-1} + u^{(k)}_{i,j+1} + h^2 f_{i,j}]$$

Two solution buffers alternate roles after each sweep. A third array stores the right-hand side. Values use double precision and row-major layout, with contiguous columns. The serial cost is $O(KN^2)$ for K sweeps, and the principal array storage is approximately $24(N+2)^2$ bytes. At N=2048, this is about 96 MiB before halo and runtime overhead.

Manufactured solutions

The continuous test uses $u^* = \sin(\pi x)\sin(\pi y)$ and $f = 2\pi^2 u^*$. Its difference from the converged discrete solution measures second-order discretization error. Separate discrete sine and asymmetric mixed-mode problems use analytically derived discrete eigenvalues; these isolate iteration and indexing correctness from discretization error. Zero forcing checks the initially converged case and avoids division by zero.

For a discrete mode $\sin(p\pi x)\sin(q\pi y)$, $\lambda_{p,q} = 4h^{-2}[\sin^2(p\pi h/2) + \sin^2(q\pi h/2)]$. From a zero initial iterate, its exact Jacobi amplitude after K sweeps is $1 - (1 - h^2\lambda_{p,q}/4)^K$. Superposing unequal (p,q) modes gives an independent finite-iteration reference that detects axis swaps and halo/indexing errors.

Stopping and timing

The relative residual is $||f - A_h u||_2 / ||f||_2$, with an absolute residual when the forcing norm is zero. Each rank forms a local squared sum; MPI_Allreduce sums before taking the square root. Convergence mode checks the initial iterate and then the requested intervals. Fixed mode always performs exactly K iterations, even when a tolerance happens to be met. A check is forced at the final requested iteration.

The solve timer includes sweeps, halo exchanges, buffer swaps, scheduled residual evaluations, and their reductions. It excludes allocation, right-hand-side initialization and norm, final diagnostics, optional gathering, and file output. MPI reports the maximum rank elapsed time, after one starting barrier. Serial and standalone OpenMP use the same timer scope. Job duration and MPI process startup time are excluded.

Why parallelization is challenging

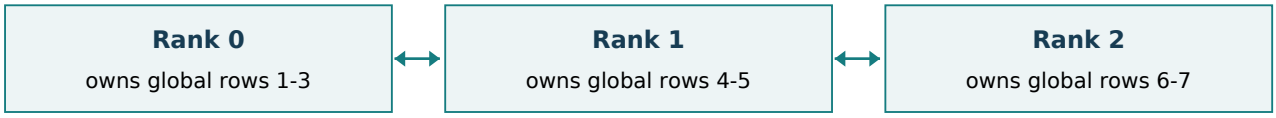
Iterations depend on the previous global state. Halo messages, global convergence reductions, synchronization, load balance, and memory locality affect performance. The nonblocking version must compute only rows whose inputs are already valid, and must complete sends before old storage can be reused. The problem is therefore not embarrassingly parallel.

2. Parallel design decisions

MPI decomposition and exchange

R MPI ranks own contiguous row slabs. Quotient/remainder partitioning gives the first N mod R ranks one extra row; ownership differs by at most one row. Each slab keeps an upper and lower ghost row. Global row offsets preserve the manufactured forcing, and MPI_PROC_NULL preserves physical boundaries. The implementation rejects R>N rather than creating empty owners.

The Sendrecv method exchanges two rows before updating the entire slab. The overlap method posts two receives and two sends, updates interior rows 2 through rows-1, waits for all four requests, and updates the boundary rows. One- and two-row slabs have explicit guards, preventing repeated updates. Current-solution ghosts are exchanged again before residual evaluation; this straightforward implementation retains some repeated communication.



Example: N=7, R=3. Adjacent owners exchange boundary rows into ghost storage.

OpenMP integration

The MPI driver accepts update, residual, and norm kernels through a LocalKernels interface. The OpenMP kernels distribute outer rows with schedule(static), retain contiguous inner-column accesses, and use reductions for local sums. Checks run before the parallel region; updates write distinct output cells. Dynamic teams are disabled, and the reported team size is probed in a real parallel region and checked across ranks.

The hybrid driver requests MPI_THREAD_FUNNELED and verifies the provided level. All MPI operations occur on the initializing thread outside OpenMP regions, consistent with the Open MPI interface [1]. Loop worksharing has an end barrier [2]. This avoids concurrent MPI calls from worker threads but still incurs a parallel-region entry and exit per kernel call.

Expected tradeoffs

Each rank performs about N^2/R updates and exchanges up to two N-element rows per sweep. Increasing R reduces compute per rank while leaving message length unchanged; global residual reductions add latency. At fixed P=R x T, fewer ranks reduce halo boundaries, but more threads add barriers and can share memory bandwidth. The nonblocking path can hide transfer time only when enough interior work exists and the MPI runtime makes progress. Its separate interior/boundary OpenMP calls can cost more than they save.

We do not use persistent OpenMP teams, explicit SIMD, parallel first-touch, a 2D process grid, multigrid, or asynchronous convergence. Array allocation and initialization are serial within each rank. These choices keep the baseline auditable and provide concrete explanations for saturation, without claiming to have isolated each mechanism experimentally.

3. Correctness validation

Validation on Vanda	Observed result
CTest suite	9 / 9 passed
Mixed OpenMP / hybrid matrix	294 / 294 oracle cases; 5 negative cases
MPI-only matrix	124 / 124 cases; 4 negative cases
Bound hybrid smoke runs	Sendrecv and overlap, 2 ranks x 2 threads passed

The matrices overlap; these counts are not independent samples to be added. They cover one/two-row slabs, uneven ownership, asymmetric forcing, K=0 and odd/even K, zero forcing, convergence cadence, halo boundaries, full-field comparison to independent dense/modal oracles, and independent residual checks. Full fields use an absolute-plus-relative tolerance of 1e-10 + 1e-10 |reference|. The performance collector separately checks executable identity, ranks, actual threads, communication mode, finite diagnostics, stopping rules, and exit status.

The first 4-CPU pilot failed: a 12-thread correctness case timed out, followed by an unhelpful missing-result exception. The exception handling was fixed without changing the numerical kernels or tolerances. The successful pilot used 16 CPUs and passive OpenMP waiting. A later performance-preparation job failed on a Python probe syntax error before measurements; its logs are retained. Neither failed job contributes timing samples.

Discretization accuracy and portable reproduction

Interior grid N	Maximum error in continuous-sine solution	Error ratio
7	1.295075e-02	-
15	3.218964e-03	4.0233
31	8.035777e-04	4.0058

Serial manufactured-solution refinement, recorded in the later Linux verification. Halving h reduces the maximum error by approximately four, consistent with the second-order spatial stencil. These error values are correctness evidence, not additional Vanda timing samples.

A subsequent portable verification expanded the mixed matrix to 390 cases plus five negative cases, including the missing OpenMP-kernel layouts 1 x 4, 2 x 1, and 3 x 1 under both exchange methods. Fresh Ubuntu builds and GitHub x86_64 CI passed all four configurations: serial (6 CTest groups), MPI (7), OpenMP (8), and hybrid (9). A source-package extraction was independently rebuilt and checked. The measured solver sources and CMake remain identical to the Vanda revision; later test counts must not be substituted for the historical 294 Vanda cases.

Reproducibility records: docs/A/results/A_1104/local, package, and remote. The independent archive audit checks all 180 launches, 150 measured samples, 30 configurations, original evidence archive hashes, and the statistics used below.

4. Experimental method

Measurement environment	Recorded value
Platform / allocation	Vanda; 1 node; 16 assigned physical cores
CPU	Intel(R) Xeon(R) Platinum 8452Y
Node physical cores	72
Toolchain	GCC 12.3.0; Open MPI 4.1.8; CMake 3.26.3
Compilation	C++17, Release -O3 -DNDEBUG; OpenMP -fopenmp; no fast-math
Repetitions	1 warmup + 5 measured, sequential case order
Formal OpenMP environment	DYNAMIC=FALSE; PLACES=cores; PROC_BIND=close; WAIT_POLICY=ACTIVE

Workloads and comparisons

Study	Count	Work held comparable	Reference
Strong	11	Fixed N=2048; K=2600	Serial executable
Weak	8	N=round(512 sqrt(P)); K=2600	MPI, one rank
Ratio	5	N=2048; 16 total cores	16 ranks x 1 thread
Communication	2	N=2048; 4 ranks x 4 threads	Sendrecv exchange
Solution	4	N=127; tolerance 1e-8	Serial, same tolerance

Thirty configurations x (one warmup + five measured launches) give 180 launches and 150 formal samples. Fixed-sweep studies use a residual check every 10 iterations. Strong efficiency is S(P)/P relative to the serial median; weak efficiency is t(MPI,1)/t(P). Fixed-core and communication comparisons are runtime ratios at equal resources, rather than serial scaling efficiencies.

Placement, timing, and experimental controls

Source was exported from the fixed Git revision and compiled on a Vanda compute node, then the same executable hashes were verified for measurement. The export contains no .git directory; the archive SHA256 and recorded source/binary hashes establish identity. PBS routes the auto queue to batch_cpu. Actual memory allocation is recorded in scheduler.txt and may exceed the requested minimum. No GPUs are used.

The launcher uses --map-by slot:PE=T --bind-to core --report-bindings [3]. Before timing, a real MPI probe for every R:T layout confirms exactly T physical cores per rank, no inter-rank overlap, and containment within the PBS CPU set. Standalone serial is constrained to that allocation but is not pinned to one individual core; OpenMP applies its own core placement. Thus this is a single-allocation comparison, not a fully controlled NUMA or frequency experiment.

5. Strong scaling and rank/thread balance

For strong scaling, N=2048, K=2600, and the residual is checked every 10 iterations. K was selected using the successful pilot serial calibration (5.779 s for 1000 iterations) to target approximately 15 seconds serial work. Speedup S(P)=median(t_serial)/median(t_P); efficiency E(P)=S(P)/P. Five repetitions give a median and observed min-max range, not a confidence interval.

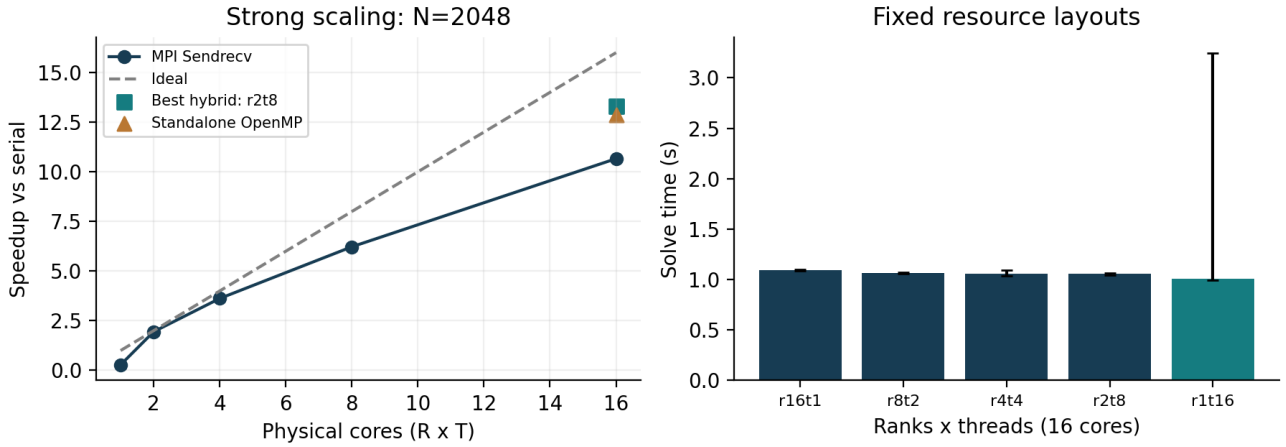


Figure 1. Strong-scaling speedup and fixed-core layout runtime. Bars show medians; whiskers show the measured minimum and maximum. MPI points use one thread per rank; the best hybrid and standalone OpenMP are also shown.

Layout	N	R x T	Median (s)	Range (s)	Speedup / E
serial	2048	1 x 1	14.276	14.119 - 15.013	1.00x / 100.0%
mpi1	2048	1 x 1	50.830	14.363 - 51.068	0.28x / 28.1%
mpi2	2048	2 x 1	7.334	7.324 - 7.426	1.95x / 97.3%
mpi4	2048	4 x 1	3.943	3.852 - 4.019	3.62x / 90.5%
mpi8	2048	8 x 1	2.294	2.199 - 2.326	6.22x / 77.8%
mpi16	2048	16 x 1	1.339	1.285 - 1.361	10.66x / 66.6%
r8t2	2048	8 x 2	1.247	1.234 - 1.269	11.45x / 71.6%
r4t4	2048	4 x 4	1.125	1.101 - 1.183	12.69x / 79.3%
r2t8	2048	2 x 8	1.073	1.050 - 1.117	13.31x / 83.2%
r1t16	2048	1 x 16	1.149	1.007 - 3.260	12.42x / 77.6%
openmp16	2048	1 x 16	1.110	1.091 - 1.149	12.86x / 80.4%

The fastest tested strong configuration is r2t8: 1.073 s, 13.31x speedup, and 83.2% efficiency. In the separate 16-core layout study, r1t16 is fastest at 1.009 s. Repeated configurations belong to different sequential study blocks, so their medians need not be identical. Cross-block variation should temper claims of small differences.

The one-rank MPI samples span 14.363 to 51.068 s, despite identical commands and successful validation. The cause was not isolated, so this anomaly must not be interpreted as a stable MPI overhead. No samples were removed. The R=1, T=16 layout also has a wide range; the ranking of nearby layout medians is tentative.

6. Weak scaling and communication method

Weak scaling keeps approximately 512 squared interior points per core, using N=round(512 sqrt(P)) and the same K=2600. The maximum rounding error in work per core is 0.021%. Weak efficiency is median(t_MPI,1)/median(t_P). It measures fixed-sweep execution, not equivalent PDE convergence as the grid is refined.

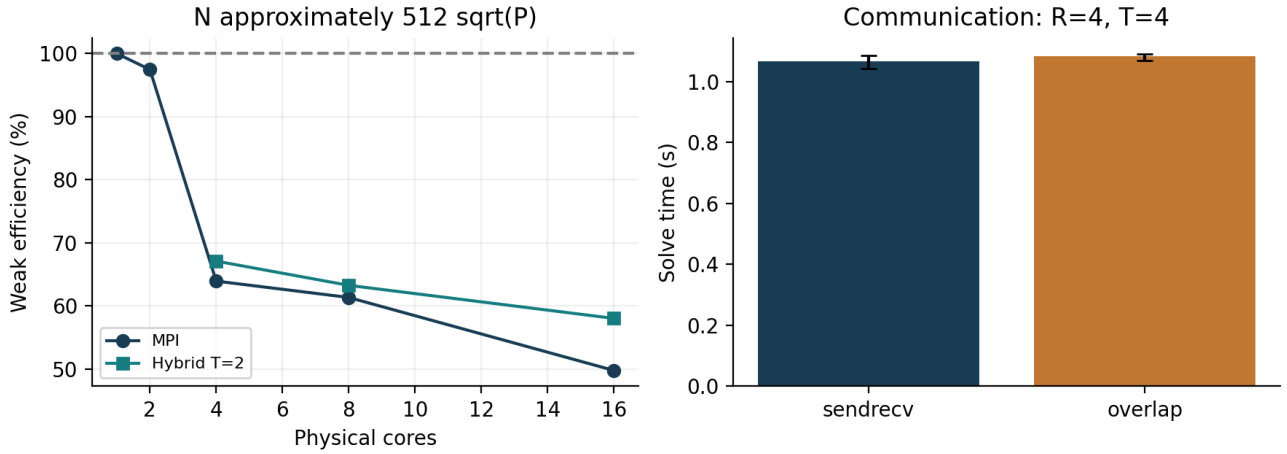


Figure 2. Weak efficiency relative to the one-rank MPI baseline, and the matched Sendrecv/overlap comparison. Communication bars show median and min-max solve times at the same R:T layout.

Layout	N	R x T	Median (s)	Range (s)	Speedup / E
mpi1	512	1 x 1	0.6204	0.6112 - 0.6278	100.0% weak
mpi2	724	2 x 1	0.6365	0.6315 - 0.6408	97.5% weak
mpi4	1024	4 x 1	0.9705	0.9607 - 0.9790	63.9% weak
mpi8	1448	8 x 1	1.011	1.001 - 1.044	61.3% weak
mpi16	2048	16 x 1	1.247	1.241 - 1.252	49.8% weak
hybrid4	1024	2 x 2	0.9246	0.9238 - 0.9262	67.1% weak
hybrid8	1448	4 x 2	0.9807	0.6987 - 1.001	63.3% weak
hybrid16	2048	8 x 2	1.069	1.062 - 1.095	58.0% weak

For the communication comparison, Sendrecv takes 1.065 s and overlap takes 1.083 s; the blocking/overlap ratio is 0.983. A ratio above one favors overlap. Nonblocking communication is not evidence by itself that communication and computation progressed simultaneously. The experiment measures complete solve time and does not collect a communication timeline.

The two observed timing ranges overlap. This run does not demonstrate a robust performance advantage for nonblocking exchange. More repetitions with interleaved case order would be needed to assess a small median difference.

At 16 cores, weak efficiency is 49.8% for MPI and 58.0% for the T=2 hybrid. The overlap median is 1.7% higher than Sendrecv. These single-node results combine memory, cache, synchronization, and communication costs. More MPI ranks shrink the independent interior; more threads increase region coordination. No bandwidth counters or NUMA experiment were collected, so these are possible mechanisms rather than established hardware causes.

7. Time to solution and conclusions

A separate N=127 continuous-sine problem runs in convergence mode with tolerance 1e-8, check interval 20, and a 200000-iteration cap. These runs compare the same discrete problem and stopping threshold. The table reports actual work and residuals from all measured samples, rather than the iteration cap.

Layout	Median s	Actual iterations	Max final rel. residual	Speedup
serial	0.5686	61160	9.977e-09	1.00x
mpi16	0.2133	61160	9.977e-09	2.67x
r4t4	0.4120	61160	9.977e-09	1.38x
openmp16	0.3249	61160	9.977e-09	1.75x

Small-grid time to solution can be dominated by coordination, so it need not follow the large fixed-work ranking. For Jacobi, slow damping of low-frequency modes also means that finer-grid solution time is much larger than a short fixed-iteration timing. The large-grid scaling runs are intentionally not presented as converged solutions.

Conclusions and limits

The implementation is numerically validated across serial, MPI, standalone OpenMP, and hybrid execution. On the tested Vanda allocation, r2t8 gives the lowest strong-study median, while r1t16 leads the fixed-16-core comparison. The data demonstrate that the rank/thread balance and communication path are experimental choices, not automatic improvements. All conclusions apply to these grid sizes, source revision, compiler, placement, and one CPU node.

We have not measured multi-node scaling or GPU execution. Open MPI logs warnings about unavailable PBS/UCX plugins; single-node execution and actual rank affinity were verified, but inter-node placement and network overlap have not been validated. The node is not reserved in its entirety. Case order is sequential, samples are limited to five, and no hardware counters or frequency controls are used. Follow-up work should test multiple nodes, randomized study order, parallel first-touch, persistent thread teams, and a 2D decomposition.

All inputs are manufactured numerical problems. There is no real dataset from which to draw application-domain conclusions. The results concern solver correctness and parallel execution efficiency.

AI usage

OpenAI Codex assisted the team with task breakdown, C++ serial/MPI/OpenMP implementation and integration, build configuration, independent-oracle tooling, automated tests, PBS scripts, experiment collection, plots, documentation, and this report. Repository records in docs/A/ai_usage.md, docs/B/ai_usage.md, and docs/C/ai_usage.md retain stage-specific use. Measurements shown here are actual program outputs; AI did not supply synthetic timing data. Verification included independent dense/modal references, full-field and residual checks, real Linux builds, CI execution, archive/statistics audits, and page-by-page PDF inspection. AI assistance covered both code and writing; measured performance values were retained without fabrication. This disclosure does not claim completed manual sign-off by every member.

8. Reproduction and evidence index

The root README describes the four executables and local build/test commands. The cluster scripts contain PBS requests, compute-node compilation, correctness validation, calibration, affinity checks, collection, and aggregation. Use a project allocation authorized for your own account; account-specific uploaded filenames are retained in submitted_script.pbs.

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release \
  -DENABLE_MPI=ON -DENABLE_OPENMP=ON
cmake --build build --parallel 2
ctest --test-dir build --output-on-failure

OMP_NUM_THREADS=4 mpiexec --map-by slot:PE=4 \
  --bind-to core -n 4 build/poisson_hybrid \
  --n 2048 --mode fixed --iterations 2600 \
  --check-every 10 --communication sendrecv
```

Run from the submission directory. Install GCC, CMake, Python 3, and Open MPI development/runtime tools first. Backslashes above continue shell commands. Quick correctness runs on small machines should set OMP_WAIT_POLICY=PASSIVE and OMP_DYNAMIC=FALSE; do not reuse that setting as the formal ACTIVE timing environment. The archived collection/config.json is authoritative for the reported experiments.

Evidence item	Purpose
source_archive.json	Source revision and uploaded archive SHA256
submitted_script.pbs / preparation.py	Exact job and matrix generator used
job_sources.sha256	Hashes of experiment scripts
scheduler.txt / job.log	Actual scheduler resources, node, modules, exit status
collection/manifest.json	Compiler/runtime versions; source and executable hashes
collection/config.json	Workloads, repeats, placement, affinity probe evidence
collection/run-*/	Commands, stdout, stderr, result, and verdict for each launch
summary/summary.json and .csv	Median, min, max, speedup, efficiency
Pilot evidence archive	Validation matrices, residual checks, calibration and binding logs

Fixed source archive SHA256:
05493d35200bb03f5f69b9fccbae36318f812dc1839194d23975d2d42a9ae19d

References

[1] Open MPI 4.1.8, MPI_Init_thread documentation. Official reference. Thread-support semantics; accessed 3 November 2026.

[2] OpenMP API 5.0, section 2.9.2, Worksharing-Loop Construct. Official specification. Cited for basic loop semantics, not as a claim of runtime version.

[3] Open MPI 4.1.8, mpirun documentation. Official reference. Rank mapping and core binding.

[4] Project source: docs/A/algorithm.md, docs/B/mpi_design.md, src/solver_mpi.cpp, src/kernel_openmp.cpp, and the validation scripts at the fixed revision above. Numerical equations and implementation descriptions are derived directly from these sources.